

Содержание

Содержание.....	3
Введение.....	4
1. Обзор текущего состояния предмета исследования.....	7
1.1. Зарубежные исследования.....	7
1.2. Отечественные исследования.....	9
1.3. Общая характеристика проблемы исследования.....	10
2. Обзор технологий и методов решения проблемы.....	12
2.1. Основные методы решения проблемы и их характеристики.....	12
2.2. Основные технологии решения проблемы и их характеристики.....	14
2.3. Дополнительные технологии решения проблемы и их характеристики.....	21
2.4. Алгоритм выбора методов и технологий решения проблемы.....	25
3. Формирование требований к решению проблемы.....	28
3.1. Общая характеристика предлагаемого решения.....	28
3.2. Функциональные требования к предлагаемому решению.....	31
3.3. Нефункциональные требования к предлагаемому решению.....	32
3.4. Архитектурная модель системы.....	34
Заключение.....	37
Список использованных источников.....	39

Введение

Развитие технологий обработки данных и искусственного интеллекта приводит к появлению новых подходов к поиску и анализу информации. Современные программные системы ориентированы на работу с большими массивами данных, которые отличаются по структуре, способу хранения и семантическому содержанию. Важной задачей становится создание решений, способных объединять текстовые, визуальные и аудио данные в едином поисковом пространстве, обеспечивая высокую точность результатов при сохранении приемлемого времени отклика. Одними из основных направлений развития в данной области являются методы машинного обучения и обработки естественного языка, основанные на больших языковых моделях (Large Language Models [1]) или визуально лингвистических моделях (Vision Language Models [2]).

Актуальность исследования обусловлена стремительным ростом объёмов разнородной информации, обрабатываемой современными информационными системами. Значительная часть таких данных представлена в виде мультимодальных источников: текстовых документов, изображений, видео, аудио, табличных структур, данных с датчиков, технической и нормативной документации. Традиционные методы полнотекстового поиска оказываются недостаточно эффективными в условиях, когда требуется обнаружение семантических связей между фрагментами данных, принадлежащих к различным типам модальностей. В связи с этим возникает необходимость разработки технологий, обеспечивающих интеллектуальный поиск и анализ мультимодальной информации.

Объектом исследования являются Retrieval-Augmented Generation модели и векторные хранилища как инструмент для поиска и анализа информации.

Предмет исследования — технологии мультимодального поиска, основанные на использовании Retrieval-Augmented Generation архитектур, включая процессы формирования эмбеддингов, хранение векторных представлений и генерацию естественно-языковых ответов.

Цель данной научно-исследовательской работы заключается в теоретическом анализе методов и технологий обработки мультимодальных данных и разработке требований к веб-приложению, предназначенному для мультимодального поиска и анализа данных на основе Retrieval-Augmented Generation архитектуры и модулей генеративных языковых моделей.

Для достижения поставленной цели необходимо **решить следующие задачи**:

1. Провести обзор отечественных и зарубежных публикаций, посвящённых вопросам мультимодального поиска, векторного представления данных и применения Retrieval-Augmented Generation архитектур.
2. Выполнить сравнительный анализ современных технологий разработки серверных и клиентских приложений для реализации систем интеллектуального поиска.
3. Сформировать функциональные и нефункциональные требования к проектируемому веб-приложению.
4. Определить архитектурные особенности предлагаемого решения, включая структуру данных, интеграционные механизмы и критерии масштабируемости.

Проблема исследования заключается в том, что генеративные архитектуры позволяют осуществлять осмысленные ответы на

естественно-языковые или визуальные запросы пользователя, однако они лишены механизма доступа к внешним источникам знаний. В условиях отсутствия контекстной информации наблюдается повышенная вероятность генерации недостоверных ответов — галлюцинаций.

Теоретическая значимость исследования заключается в систематизации фундаментальных методов обработки мультимодальных данных, сравнительном анализе технологий векторного поиска и формализации комплекса требований к веб-приложениям данного класса.

Практическая значимость заключается в создании методической основы для реализации полнофункционального программного решения, ориентированного на мультимодальный поиск и анализ данных.

Структура отчёта научно-исследовательской работы соответствует требованиям методических рекомендаций и включает введение, три аналитических раздела, заключение и список использованных источников.

В первой главе рассматриваются теоретические и методологические основы мультимодального поиска и архитектур Retrieval-Augmented Generation. Проводится обзор зарубежных и отечественных исследований в области семантического и мультимодального поиска.

Вторая глава посвящена анализу и сравнительной оценке технологий, применяемых для реализации систем мультимодального поиска. В ней рассматриваются методы векторизации и семантического поиска, стек серверных и клиентских технологий.

Третья глава ориентирована на формирование требований к программному решению и описание его архитектурной модели. В рамках данной главы определяются функциональные и нефункциональные требования к системе.

1. Обзор текущего состояния предмета исследования

1.1. Зарубежные исследования

Современные зарубежные исследования в области мультимодального поиска характеризуются активным развитием технологий Retrieval-Augmented Generation, далее (RAG) [3], и интеграции генеративных моделей с системами извлечения информации. Согласно аналитическим публикациям крупных технологических компаний и академических лабораторий (IBM [6], NVIDIA [7], HuggingFace [8], Langchain [9]), ключевым направлением является соединение методов информационного поиска, обработки естественного языка и мультимодальных моделей глубокого обучения в единые гибридные конвейеры.

Одним из наиболее заметных достижений является расширение классического RAG-подхода за счёт включения мультимодальных данных — изображений, таблиц, PDF-документов, структурированных и неструктурированных источников. В зарубежных работах подчёркивается, что такие системы способны сочетать механизмы векторного поиска с визуально-лингвистическими моделями, что значительно повышает качество интерпретации данных и снижает долю ошибок генерации. IBM определяет мультимодальный RAG как перспективное направление, позволяющее расширить доступ моделей к внешним источникам знаний без необходимости повторной тренировки крупных нейронных сетей [6].

Особое внимание уделяется методам построения и оптимизации векторных хранилищ [4]. Зарубежные платформы используют структуры графов ближайших соседей, включая HNSW (Hierarchical Navigable Small World), обеспечивающие эффективный поиск в высокоразмерных базах. NVIDIA указывает, что использование графовых алгоритмов для индексации

эмбедингов [5] является критическим условием высокой производительности системы и масштабируемости при работе с большими мультимодальными коллекциями данных [7].

Отдельную группу исследований составляют работы, посвящённые мультимодальным моделям глубокого обучения, способным извлекать признаки одновременно из текста и из изображений. Согласно материалам HuggingFace, современные модели, такие как BLIP-2, Flamingo, Kosmos-1, Gemini и GPT-4, формируют контекстно устойчивые эмбединги, позволяющие корректно сопоставлять различные типы данных [8]. Это создаёт основу для построения универсальных систем поиска, способных интерпретировать визуальные объекты, диаграммы, графические элементы, текстовые фрагменты и цифровые документы в едином признаковом пространстве.

Важным направлением зарубежных публикаций является разработка гибридных методов ранжирования, сочетания классического полнотекстового поиска (BM25, TF-IDF) и векторного поиска по эмбедингам. Подобные методы, описанные в руководствах по применению RAG-технологий LangChain, позволяют повысить точность выдачи и минимизировать влияние ошибок извлечения ключевых слов [9]. Комбинация поисковых стратегий обеспечивает значительный прирост качества генерации ответов при сложных и плохо структурированных запросах.

Таким образом, анализ зарубежных источников позволяет выделить следующие устойчивые направления развития мультимодальных систем на основе RAG:

- интеграция генеративных моделей с внешними хранилищами данных;
- развитие мультимодальных моделей глубокого обучения и визуально-лингвистических архитектур;

- использование высокопроизводительных векторных хранилищ на основе графовых алгоритмов HNSW;
- применение гибридных механизмов ранжирования для увеличения точности поиска;
- масштабирование рантайм-конвейеров для обработки больших корпусов данных.

Данные тенденции подтверждают высокий интерес научного сообщества и индустрии к созданию систем, способных не только извлекать релевантную информацию, но и формировать осмысленные ответы на естественно-языковые запросы в условиях разнородных мультимодальных данных.

1.2 Отечественные исследования

В России задачи мультимодального поиска, генерации ответов и архитектуры RAG находятся на этапе активного развития. Для отечественных исследований характерна ориентация на применения технологий в условиях корпоративного документооборота, анализа нормативно-правовых актов, систем мониторинга и информационного сопровождения предприятий.

Особое внимание уделяется созданию языковых моделей, адаптированных к русскому языку и специализированным предметным областям. Сбер AI предлагает архитектуру GigaChat, выполняющую обработку текстов и генерацию ответов с учётом морфологических особенностей русского языка [10]. Яндекс развивает собственную линейку моделей YaGPT, ориентированных на задачи анализа документов и информационного поиска [11].

В работах российских исследователей отмечается, что применение векторных представлений данных и методов семантического поиска

существенно повышает эффективность работы с большими массивами текста, характеристическими для правового анализа, медицинской документации и технических регламентов. Кроме того, исследования показывают, что применение RAG-архитектур позволяет существенно снизить количество ошибок в ответах, генерируемых языковыми моделями, благодаря привязке к внешним источникам данных.

Актуальные направления отечественных исследований включают:

- разработку и обучение эмбединговых моделей для русского языка;
- создание специализированных корпусов данных для обучения моделей мультимодального поиска;
- интеграцию RAG-систем с существующими корпоративными базами данных и службами документооборота.

1.3 Общая характеристика проблемы исследования

Анализ зарубежных и отечественных исследований позволяет сформулировать основные проблемы, препятствующие эффективной реализации систем мультимодального поиска и генерации ответов.

Во-первых, наблюдается отсутствие унифицированных подходов к представлению мультимодальных данных. Текстовые документы, изображения и табличные структуры требуют различных механизмов предобработки и извлечения признаков, что усложняет формирование единого пространства признаков. Эта проблема усугубляется необходимостью сохранения внутренней структуры источников данных (разметки документов, иерархии разделов, связей между объектами) при одновременном обеспечении быстрого поиска в условиях больших объёмов информации.

Во-вторых, существующие генеративные модели без доступа к внешним источникам данных склонны к формированию недостоверных ответов. Это особенно критично в области корпоративных информационных систем, где точность и воспроизводимость выдаваемых сведений являются ключевыми факторами эффективности работы и доверия пользователей к системе. Возникает необходимость сочетать мощность генеративных моделей с контролируемым доступом к проверенным данным и механизмами верификации результата.

В-третьих, обеспечение масштабируемости систем мультимодального поиска требует наличия инструментов горизонтального масштабирования и распределённой обработки. При увеличении объёма данных и числа одновременных запросов необходимо поддерживать стабильное время отклика и высокую степень доступности сервисов, что предполагает использование специализированных решений для векторного поиска, балансировки нагрузки и отказоустойчивой инфраструктуры.

Дополнительной проблемой является сложность интеграции подобных систем в существующую информационную инфраструктуру организаций. Требуется обеспечить совместимость с уже используемыми хранилищами данных, сервисами аутентификации и системами мониторинга, а также соблюдение требований информационной безопасности и законодательства в области защиты персональных данных. Кроме того, остаётся нерешённой задача выработки единых методик оценки качества мультимодального поиска и сгенерированных ответов, что затрудняет сопоставление различных архитектур и технологий между собой и выбор оптимальных решений для практического внедрения.

2. Обзор технологий и методов решения проблемы

2.1. Основные методы решения проблемы и их характеристики

Мультимодальный поиск предполагает обработку разнородных данных с целью извлечения семантически релевантной информации по естественно-языковому запросу. Наиболее эффективным подходом к решению данной задачи является использование векторных представлений данных, формируемых моделями глубокого обучения. Использование векторных признаков позволяет преобразовать объекты различных типов (тексты, изображения, видео, аудио, табличные структуры) в единую математическую форму — многомерные векторы фиксированной размерности.

Основным математическим инструментом для вычисления семантической близости между объектами выступает метрика расстояния между векторами. Наиболее широко распространены метрики косинусного расстояния, евклидового расстояния и манхэттенской метрики. Выбор метрики определяется задачами конкретного приложения и размерностью пространства признаков.

При этом важным элементом является процесс векторизации данных (embedding). Векторизация текста осуществляется с помощью предобученных языковых моделей, таких как OpenAI Text Embeddings [13], а также моделей GigaChat Embeddings и YaGPT от отечественных компаний [5; 12]. Векторизация изображений выполняется с использованием моделей, основанных на архитектуре CLIP, BLIP-2 и аналогичных моделей для извлечения признаков изображения [14; 15].

Дополнительным направлением является использование методов гибридного поиска. В данном случае комбинируются результаты

полнотекстового ранжирования (BM25) и векторного поиска, что позволяет учитывать как точное совпадение ключевых слов, так и семантическую близость.

В условиях разработки и эксплуатации систем важную роль играет обеспечение непрерывности процессов развертывания, тестирования и доставки программных компонентов. Технологии CI/CD представляют собой методологию, направленную на автоматизацию жизненного цикла программного обеспечения, начиная с проверки изменений исходного кода и заканчивая развёртыванием рабочих версий системы на сервере [16].

CI (Continuous Integration) — это практика регулярного объединения изменений в центральной репозитории и автоматического выполнения тестов. CI позволяет выявлять ошибки на ранних этапах разработки, снижать вероятность конфликтов между разработчиками и обеспечивать стабильность кода.

CD, в свою очередь делится на два понятия (Continuous Delivery и Continuous Deployment).

Continuous Delivery — это автоматизация процесса подготовки приложения к развёртыванию. На данном этапе выполняется сборка, тестирование и упаковка программных компонентов, что позволяет сократить время между финализацией кода и внедрением обновлений в рабочую среду.

Continuous Deployment предполагает автоматическую установку обновлений в production-среду без участия разработчика. Это актуально для систем, требующих высокой доступности и минимальных простоев.

В системах мультимодального поиска, представляющих собой комплекс программных модулей (backend, vector storage, модели эмбедингов,

клиентский интерфейс, балансировщики нагрузки), применение CI/CD обеспечивает следующие преимущества:

1. **Гарантия стабильности RAG-пайплайна** — при каждом изменении в логике выполняются автотесты и регрессионная проверка.
2. **Автоматическое обновление эмбединговых моделей** — изменения параметров векторизации могут быть протестированы и развернуты без ручного вмешательства.
3. **Быстрое внедрение обновлений API** — API-сервисы могут масштабироваться и обновляться без остановки системы.
4. **Контроль качества данных** — автоматические тесты могут включать проверку корректности загрузки данных в базу данных.
5. **Минимизация риска при распределённых вычислениях** — CI/CD позволяет протестировать сценарии масштабирования до развертывания.

2.2. Основные технологии решения проблемы и их характеристики

Технологический стек программного решения должен обеспечивать целостность системы мультимодального поиска, включая процессы векторизации данных, хранение векторных представлений, управление вычислительными ресурсами и взаимодействие с генеративными моделями. Важным критерием выбора технологий является не только функциональность отдельных инструментов, но и степень их интегрируемости, расширяемости и совместимости в рамках единой архитектуры.

Для реализации серверной части целесообразно использование языка программирования **Python**, обладающего развитой экосистемой библиотек машинного обучения. Применение Python обеспечивает доступ к широкому спектру фреймворков, предназначенных для работы с эмбединговыми

моделями, генеративными архитектурами и векторными хранилищами, что позволяет минимизировать затраты на интеграцию отдельных компонентов системы [17].

Ключевым элементом серверного взаимодействия выступает фреймворк **FastAPI** [18], ориентированный на создание высокопроизводительных REST API [19] и WebSocket-интерфейсов [20]. В отличие от традиционных решений, таких как Flask [21] или Django REST Framework [22], FastAPI поддерживает асинхронную обработку запросов, что существенно снижает задержку и повышает скорость реакции системы при высокой нагрузке. Данный фреймворк также предоставляет встроенные средства автоматической документации API (Swagger/OpenAPI), что упрощает интеграцию клиентских приложений [23].

Для реализации механизмов Retrieval-Augmented Generation применяется инструмент **LangChain**, обеспечивающий формирование последовательностей обработки данных, включающих этапы векторизации, поиска релевантных документов и генерацию естественно-языкового ответа [24]. Выбор данной технологии обусловлен наличием встроенной поддержки генеративных моделей, модулей взаимодействия с внешними провайдерами (OpenAI, HuggingFace [25; 26]), а также реализацией шаблонов построения сложных RAG-конвейеров, включая retrieval, reranking и пост-обработку результатов.

LangChain является предпочтительным решением для систем мультимодального поиска благодаря следующим преимуществам [27]:

- унифицированный интерфейс взаимодействия с различными источниками данных;
- возможность использования нескольких стратегий поиска (dense search, hybrid search, multi-query);

- поддержка сохранения контекста и цепочек рассуждений (chain-of-thought).

Векторные базы данных являются одним из ключевых элементов архитектуры мультимодального поиска. Они предназначены для хранения многомерных представлений данных и обеспечения поиска ближайших соседей в высокоразмерных пространствах. Наиболее известными системами являются Qdrant и pgvector [28; 29].

Таблица 2.1 — Сравнение Qdrant и pgvector по ключевым характеристикам

Критерий	Qdrant	pgvector	Вывод
Алгоритм поиска	HNSW	k-NN [30]	Qdrant быстрее при больших коллекциях
Масштабируемость	Кластеры, шардинг	Только через PostgreSQL [31]	Qdrant подходит для системы с ростом данных
API	REST, gRPC [32]	SQL [33]	Qdrant проще интегрировать с LangChain
Формат хранения	Vector + payload	Table rows	Qdrant поддерживает гибридный поиск «вектор + метаданные»
Производительность	10^6 – 10^8 объектов	10^4 – 10^5 объектов	Qdrant эффективен для мультимодальных систем

На основании проведённого анализа наиболее предпочтительным решением для реализации системы мультимодального поиска является **Qdrant**.

Для выполнения параллельной обработки больших корпусов данных и реализации распределённой RAG-инфраструктуры используется кластер вычислительных узлов, управляемый библиотекой **Ray** [34]. Данная технология позволяет автоматически распараллеливать задачи векторизации, индексации и построения эмбеддингов, сохраняя при этом единый интерфейс вызова функций.

Для оценки технологии масштабирования выполнен сравнительный анализ:

Таблица 2.2 — Сравнение технологий масштабирования

Технология	Назначение	Масштабируемость	Уровень вмешательства
Ray	Распределённые вычисления	Высокая	Низкий
Celery [35]	Очереди задач	Средняя	Средний
Apache Spark [36]	Big Data	Очень высокая	Высокий

Выбор Ray обусловлен балансом между удобством разработки и эффективной поддержкой горизонтального масштабирования без необходимости ручной конфигурации больших вычислительных кластеров.

Для обеспечения мобильности и изолированности компонентов системы применяется контейнеризация с использованием технологии **Docker** [37], которая позволяет независимо управлять сервисами Qdrant, Ray и клиентской частью. Контейнерный подход обладает следующими преимуществами:

- снижает вероятность конфликтов зависимостей;
- обеспечивает воспроизводимость окружений;
- упрощает интеграцию с CI/CD-конвейерами.

Для управления развертыванием микросервисов используется система **Kubernetes**, предоставляющая механизмы автоматического

масштабирования, балансировки нагрузки и мониторинга состояния контейнеров [38]. Применение Kubernetes позволяет реализовать отказоустойчивую конфигурацию, при которой сбой одного узла не препятствует функционированию системы в целом.

Таблица 2.3 — Характеристика технологий серверной части

Технология	Назначение	Ключевые преимущества
Python	Реализация серверной логики, взаимодействие с моделями и обработка данных	Развитая экосистема библиотек для работы с эмбедингами, поддержка асинхронных фреймворков, низкий порог интеграции с ML-инструментами
FastAPI	Построение REST и WebSocket API	Асинхронная обработка запросов, автоматическая генерация OpenAPI-спецификации, высокая производительность при параллельной нагрузке
LangChain	Формирование RAG-цепочек и взаимодействие с генеративными моделями	Поддержка retrieval, reranking и post-processing, унификация доступа к внешним LLM и VLM, modular pipeline
Qdrant	Хранение и поиск векторных представлений	Алгоритм HNSW для быстрого поиска ближайших соседей, гибридная фильтрация по метаданным, поддержка кластеризации
Ray	Организация распределённых вычислений	Автоматическое распараллеливание операций, горизонтальное масштабирование, поддержка распределённых RAG-пайплайнов

Таблица 2.4 — Технологии контейнеризации и оркестрации

Технология	Назначение	Ключевые характеристики
------------	------------	-------------------------

Docker	Контейнеризация компонентов системы	Изоляция окружений, воспроизводимость, независимость зависимостей
Kubernetes	Управление кластером контейнеров	Автоматическое масштабирование, балансировка нагрузки, мониторинг состояния

Клиентская часть системы играет ключевую роль в обеспечении взаимодействия пользователя с веб-приложением, визуализации результатов мультимодального поиска и формировании запросов к серверной части. От состояния пользовательского интерфейса зависит не только удобство эксплуатации системы, но и общая эффективность обработки информации, поскольку корректная и интуитивная форма представления данных способствует снижению когнитивной нагрузки и ускоряет анализ полученных результатов. Для реализации интерфейса была выбрана связка современных технологий, обеспечивающих высокую производительность, гибкость и расширяемость приложения.

Важной основой клиентской части выступает язык **TypeScript**, позволяющий применять статическую типизацию для контроля взаимодействия между компонентами приложения [39]. В отличие от традиционного JavaScript [40], TypeScript предоставляет механизмы типобезопасности, что снижает вероятность логических ошибок, упрощает отладку и повышает надёжность клиентского кода при работе с асинхронными запросами к серверу. Это особенно важно в системах мультимодального поиска, где обрабатываемые данные могут иметь сложную структуру, включающую текстовые фрагменты, метаданные и векторные представления.

Фреймворк **Nuxt 4** [41], построенный на основе **Vue 3** [42], обеспечивает высокую реактивность интерфейса и поддерживает гибкие

сценарии маршрутизации и серверного рендеринга (SSR [43]). Применение Nuxt позволяет оптимизировать начальную загрузку страницы, повышает скорость отображения результатов и улучшает показатели Core Web Vitals [44], что критично для приложений, работающих с большим количеством динамического контента. Компонентный подход Vue 3 упрощает проектирование интерфейсов, позволяя реализовывать визуальные элементы как автономные блоки с чётко определённой функциональностью.

Для обеспечения единообразного и адаптивного оформления элементов пользовательского интерфейса используется **Tailwind CSS**, представляющий собой функционально-ориентированную библиотеку для быстрой стилизации [45]. Tailwind позволяет формировать дизайн на основе множества стилизующих классов, что минимизирует использование каскадных таблиц стилей вручную и ускоряет разработку интерфейса. При этом обеспечивается адаптивность интерфейса, что особенно важно для отображения результатов мультимодального поиска на устройствах с различными разрешениями экрана.

Инструментом сборки клиентской части является **Vite**, обеспечивающий высокую скорость разработки и низкое время отклика при горячей перезагрузке модулей [46]. По сравнению с классическими инструментами, такими как Webpack [47], Vite существенно ускоряет процесс компиляции и обеспечивает эффективную работу с зависимостями, что снижает время разработки и повышает качество итогового приложения. Использование Vite также позволяет легко интегрировать дополнительные инструменты, такие как ESLint [48], Prettier [49] и средства тестирования End-to-End.

Таблица 2.5 — Характеристика технологий клиентской части

Технология	Назначение	Преимущества
------------	------------	--------------

TypeScript	Типизация кода	Надёжность, снижение ошибок
Nuxt 4	Клиентский интерфейс	SSR, высокая производительность
Tailwind CSS	Стилизация компонентов	Быстрое прототипирование
Vite	Сборка проекта	Высокая скорость, простота настройки

Таким образом, представленные технологии в совокупности обеспечивают надёжность, предсказуемость кода и высокую производительность. Это позволяет реализовать удобный для пользователя интерфейс при сохранении высокой эффективности разработки и сопровождения системы.

2.3. Дополнительные технологии решения проблемы и их характеристики

Разработка системы мультимодального поиска на основе RAG-архитектуры требует использования не только основных инструментов обработки данных и серверного взаимодействия, но и дополнительных технологий, обеспечивающих контроль качества кода, оптимизацию процессов разработки, мониторинг состояния системы и автоматизацию документирования. В данном подразделе представлены вспомогательные технологии, позволяющие повысить стабильность, наблюдаемость, воспроизводимость и сопровождаемость разработанного решения.

Для эффективного управления зависимостями в среде разработки используется инструмент UV [50], представляющий собой высокопроизводительный менеджер пакетов и окружений для Python. В отличие от стандартных решений, таких как pip [51] или Poetry [52], UV обеспечивает низкое время установки пакетов, поддержку изоляции

окружений, ускоренные операции кеширования и воспроизводимость версий зависимостей. Это становится критически важным при работе с большим количеством библиотек для машинного обучения, векторной обработки данных и интеграции с LangChain.

Дополнительно применяется **Bun** [53] — современная платформа для выполнения JavaScript- и TypeScript-кода, ориентированная на высокую производительность и низкое время холодного старта. Bun значительно превосходит традиционные инструменты (Node.js и npm/yarn [54; 55; 56]) по скорости пакетного менеджмента и запуску локальных задач сборки. Благодаря этому сокращается время разработки клиентской части приложения, основанной на Nuxt 4 и Tailwind CSS, и ускоряется выполнение операций в CI/CD-конвейерах.

Для обеспечения контроля над эксплуатационными характеристиками системы используются инструменты мониторинга **Prometheus** [57] и визуализации данных **Grafana** [58]. В условиях распределённых вычислений и интенсивной работы векторного хранилища наличие аналитических механизмов для отслеживания состояния сервисов является обязательным элементом производственной инфраструктуры.

- **Prometheus** выполняет сбор и хранение метрик в формате временных рядов (time-series), фиксируя состояние отдельных компонентов системы: задержку запросов FastAPI, скорость векторного поиска в Qdrant, загрузку процессорных ядер Ray-узлов и использование оперативной памяти.
- **Grafana** предоставляет интерфейс визуализации и построения интерактивных панелей, что позволяет оперативно выявлять аномалии, отслеживать состояние RAG-цепочек и прогнозировать нагрузку при масштабировании.

Использование Prometheus и Grafana в связке позволяет осуществлять полноценный мониторинг производительности и своевременное обнаружение узких мест, повышая надёжность и отказоустойчивость системы.

С целью предотвращения ошибок на ранних этапах разработки используется инструмент **pre-commit** [59], обеспечивающий автоматическое выполнение проверок перед фиксацией изменений в систему контроля версий. В систему pre-commit включаются следующие задачи:

- статический анализ исходного кода;
- проверка форматирования;
- валидация файлов зависимостей;
- автоматическая очистка и нормализация кода.

Таким образом, pre-commit позволяет упростить соблюдение стандартов разработки и повысить качество исходного кода за счёт автоматизации рутинных задач.

Для структурирования технической документации и автоматизации её подготовки используется генератор статических сайтов **mkdocs** [60], основанный на языке разметки Markdown [61].

Преимущества mkdocs включают:

- генерацию документации в виде веб-сайта;
- встроенную поддержку версионирования документации;
- работу с множеством расширений (схемы Mermaid [62], таблицы, подсветка кода);
- совместимость с CI/CD для автоматического обновления документации при изменении исходного кода.

Документирование является важным аспектом в разработке сложных программных систем: оно снижает порог входа для новых участников проекта и способствует качественной поддержке системы на всех этапах её жизненного цикла.

Таблица 2.6 — Характеристика дополнительных технологий

Технология	Назначение	Ключевые характеристики
UV	Управление зависимостями и пакетами для среды разработки	Высокая скорость установки библиотек, оптимизация сборки Python-проектов, наличие кроссплатформенной поддержки
Bun	Среда выполнения и сборки клиентских модулей	Низкое время холодного старта, высокая производительность при работе с JavaScript/TypeScript, встроенные средства сборки
Prometheus	Сбор и хранение метрик состояния системы	Модель временных рядов, язык запросов PromQL [63], поддержка экспортеров и мониторинга микросервисов
Grafana	Визуализация метрик и построение аналитических панелей	Поддержка подключаемых источников данных, гибкие дашборды, инструменты аналитики состояния системы
pre-commit	Контроль качества исходного кода	Автоматическая проверка формата, статический анализ, выполнение тестов перед фиксацией изменений
mkdocs	Автоматизация подготовки документации	Генерация статических сайтов, структурирование технических материалов, поддержка разметки Markdown

Дополнительные технологии формируют инфраструктурный контур системы, отвечающий за контроль качества разработки, наблюдаемость и документирование. Их использование снижает операционные риски при эксплуатации программного комплекса и упрощает его дальнейшую эволюцию без существенных затрат на поддержку.

2.4. Алгоритм выбора методов и технологий решения проблемы

Алгоритм выбора методов и технологий разработки системы мультимодального поиска строится на принципах сопоставления функциональных требований, свойств данных и эксплуатационных характеристик программного решения. Данный подход предполагает принятие решений на основе анализа альтернатив по ряду критериев, включая производительность, масштабируемость, совместимость компонентов, сложность интеграции, сопровождаемость кода, а также ограничения по безопасности и стоимости владения.

На первом этапе определяется специфика данных, подлежащих обработке, с описанием их природы (текст, изображения, видео, аудио, структурированные записи), объёма и скорости обновления. На основании полученных сведений формируются требования к способам их представления и предварительной обработке, включая необходимость нормализации, очистки, аннотирования или извлечения признаков.

Следующий этап заключается в выборе методов преобразования данных в векторные представления. На данном шаге проводится оценка моделей по параметрам качества эмбедингов, вычислительной эффективности, доступности предобученных вариантов и совместимости с внешними источниками данных. Рассматриваются различные архитектурные подходы, включая статические и контекстные модели, одно- и

мультимодальные решения, а также способы получения, обновления и хранения эмбедингов.

После определения стратегии векторизации осуществляется анализ существующих решений для хранения векторных данных. На этом этапе оцениваются алгоритмы поиска ближайших соседей, доступные методы фильтрации, поддержка гибридных запросов, а также режимы масштабирования и отказоустойчивости. Сравниваются специализированные векторные базы данных и универсальные системы хранения с модульной поддержкой эмбедингов.

Определение методов генерации ответов основывается на сравнении различных архитектур генеративных моделей. Особое внимание уделяется стратегиям снижения недостоверных выводов, в том числе использованию RAG-подхода, повторного ранжирования результатов поиска и внедрению проверок соответствия источникам данных. Важными критериями являются устойчивость модели, управляемость процессом генерации, наличие интерфейсов для взаимодействия с внешними сервисами и возможность ограничения доступа к конфиденциальной информации.

Выбор технологий для реализации серверной части базируется на оценке их производительности, функциональной гибкости и поддержке параллельной обработки запросов. Рассматриваются фреймворки, методы построения API и средства достижения низкого времени ответа при высоких нагрузках. На этом этапе также учитываются возможности распределённой обработки данных, наличие инструментов профилирования, средств логирования и интеграции с системами мониторинга, что обеспечивает наблюдаемость и управляемость серверной инфраструктуры.

Формирование требований к клиентской части связано с выбором подходов к построению пользовательского интерфейса. Анализируются

средства разработки реактивных компонентов, поддержка адаптивных интерфейсов, реализация маршрутизации и способы обмена данными с сервером. Ключевыми критериями выбора являются удобство взаимодействия пользователя с системой, скорость отображения результатов, поддержка расширения функционала на последующих стадиях разработки и соответствие требованиям доступности интерфейса для различных категорий пользователей.

Отдельный этап алгоритма посвящён выбору технологий сопровождения разработки и эксплуатации системы. На данном этапе оцениваются методы контроля качества кода, инструменты документирования, системы мониторинга, средства автоматизации тестирования и развёртывания. Анализ проводится по критериям удобства интеграции с системой контроля версий, прозрачности процессов, наличия готовых практик CI/CD и поддержки промышленного внедрения решений в различных средах (тестовой, промежуточной и продуктивной).

Завершающим шагом является валидация принятых архитектурных и технологических решений на основе прототипирования и нагрузочного тестирования. На данном этапе проверяется соответствие выбранных методов и инструментов ранее сформулированным требованиям, проводится корректировка параметров и при необходимости пересмотр отдельных элементов стека.

Таким образом, алгоритм выбора методов и технологий представляет собой структурированный процесс, включающий анализ требований, сравнение альтернатив, экспериментальную проверку и обоснование решений по каждому функциональному компоненту. Такой подход позволяет сформировать оптимальный технологический стек, соответствующий целям разработки, особенностям данных и условиям эксплуатации системы мультимодального поиска.

3. Формирование требований к решению проблемы

3.1. Общая характеристика предлагаемого решения

Предлагаемое программное решение представляет собой веб-систему интеллектуального мультимодального поиска и анализа данных, основанную на архитектуре Retrieval-Augmented Generation (RAG). Система предназначена для автоматизированного извлечения информации из разнородных источников, преобразования данных в векторные форматы, хранения в специализированной векторной базе данных и формирования смысловых ответов с использованием генеративной модели. Основная идея разработки заключается в объединении методов векторного поиска и нейросетевых механизмов генерации ответов, что позволяет повысить точность, достоверность и релевантность результатов, а также снизить вероятность возникновения недостоверных выводов модели.

Архитектура решения включает несколько функциональных модулей, взаимодействующих между собой по принципу микросервисного построения. Серверная часть реализована на основе асинхронного фреймворка FastAPI, обеспечивающего высокую производительность и низкое время отклика при большом количестве параллельных запросов. Для реализации механизма построения поисковых RAG-цепочек используется библиотека LangChain, предоставляющая унифицированный интерфейс для выполнения этапов векторизации, поиска по эмбедингам и генерации ответа. В качестве системы хранения эмбедингов применяется векторная база данных Qdrant, поддерживающая алгоритм поиска ближайших соседей HNSW, гибридную фильтрацию по метаданным и горизонтальное масштабирование.

Особое внимание уделено обеспечению масштабируемости и устойчивости системы при обработке больших объёмов данных. Для распределённой обработки и параллельного выполнения задач предусмотрено использование фреймворка Ray, позволяющего запускать вычислительные процессы на нескольких узлах и эффективно распараллеливать операции векторизации, индексирования и построения эмбедингов. Контейнеризация сервисов реализована с применением технологий Docker и Kubernetes, что обеспечивает отказоустойчивость, гибкость конфигурации и независимость микросервисов при обновлениях.

Клиентская часть приложения разработана на основе Nuxt 4 и Vue 3, что позволяет обеспечить высокую реактивность интерфейса, динамическую визуализацию результатов поиска, удобство взаимодействия пользователя с системой и поддержку различных сценариев работы с мультимодальными данными. В качестве инструмента сборки используется Vite, обеспечивающий минимальное время компиляции и эффективную работу с зависимостями. Tailwind CSS применён для унифицированной стилизации интерфейса и реализации адаптивного дизайна.

Дополнительные технологии служат для обеспечения промышленного качества реализации и эксплуатации разработанного решения. Инструмент UV используется для управления зависимостями и ускоренной установки библиотек Python, в то время как Bun применяется для запуска и сборки клиентских модулей с минимальным временем отклика. Для автоматизации проверки качества кода применяется система pre-commit, выполняющая статический анализ, форматирование и валидацию файлов перед фиксацией изменений в системе контроля версий. Документирование и структурирование технических материалов выполняется с использованием mkdocs, обеспечивающего генерацию документации в виде статического веб-сайта.

Важной особенностью предлагаемого решения является обеспеченная наблюдаемость и управляемость в условиях эксплуатации. Для сбора метрик и мониторинга состояния сервисов используются Prometheus и Grafana, позволяющие отслеживать показатели производительности, выявлять аномальные нагрузки и своевременно обнаруживать потенциальные сбои. Такой подход обеспечивает возможность оперативного реагирования на изменения в поведении системы и способствует повышению её отказоустойчивости и предсказуемости работы.

К основным задачам разрабатываемой системы относятся:

- извлечение информации из разнородных источников с последующей нормализацией данных;
- преобразование данных в векторные представления и их размещение во внешнем хранилище;
- выполнение поиска по векторным коллекциям с учётом семантического сходства;
- генерация релевантных ответов на основе найденных фрагментов данных;
- снижение вероятности возникновения «галлюцинаций» генеративной модели за счёт применения RAG-подхода;
- обеспечение высокой производительности, масштабируемости и доступности программной системы;
- мониторинг технических параметров и обеспечение контроля качества разработки.

Разработанное решение представляет собой цельную архитектурную платформу для построения систем интеллектуального поиска, сочетающую алгоритмы обработки естественного языка, методы векторного поиска и инструменты промышленной эксплуатации. Такая структура обеспечивает расширяемость, сопровождаемость и интеграцию с внешними системами, что

позволяет использовать полученные результаты при разработке приложений промышленного уровня.

3.2. Функциональные требования к предлагаемому решению

Функциональные требования задают перечень операций, которые разрабатываемая система мультимодального поиска должна выполнять при штатных условиях эксплуатации.

Таблица 3.1 — Функциональные требования

Требование	Описание
Загрузка данных	Система обеспечивает возможность загрузки файлов различных типов, включая текстовые и графические форматы.
Извлечение информации	Система выполняет автоматическое извлечение текстовых фрагментов и сопутствующих метаданных из загруженных документов.
Формирование векторных представлений	Система преобразует извлечённые данные в числовые векторные представления для последующей обработки.
Размещение данных во внешнем хранилище	Система сохраняет сформированные векторные представления и связанные метаданные во внешнем хранилище.
Семантический поиск	Система обеспечивает поиск наиболее релевантных записей на основе вычисления схожести векторных представлений.
Контекстная генерация ответов	Система формирует естественно-языковой ответ на основании данных, найденных в результате поиска.

Управление доступом	Система предоставляет механизм аутентификации пользователей для ограничения несанкционированного доступа.
Ведение истории взаимодействия	Система регистрирует запросы пользователей и сохраняет результаты поиска для последующего анализа.

Совокупность зафиксированных функциональных требований охватывает полный цикл обработки данных в системе: от загрузки и извлечения информации до семантического поиска, генерации ответов и журналирования действий пользователей. Тем самым задаётся минимально необходимый набор функций, обеспечивающих реализацию мультимодального поиска на основе RAG-подхода и поддержку ключевых пользовательских сценариев. Данные требования также служат основой для последующей разработки плана тестирования, позволяя формализовать критерии проверки корректности работы отдельных модулей и системы в целом.

3.3. Нефункциональные требования к предлагаемому решению

Нефункциональные требования задают совокупность качественных характеристик разрабатываемой системы и определяют ограничения, в рамках которых должны выполняться её функциональные возможности. К таким характеристикам относятся показатели производительности, масштабируемости, надёжности, безопасности, удобства использования и соблюдения нормативных требований. В отличие от функциональных требований, описывающих, *что* именно делает система, нефункциональные требования фиксируют, *как* она должна это делать с точки зрения качества, ресурсов и условий эксплуатации. Они оказывают прямое влияние на выбор архитектурных решений и технологии реализации.

Таблица 3.2 — Нефункциональные требования

Категория	Требование
Производительность	Система должна обеспечивать среднее время ответа на пользовательский запрос не более 1,5 секунды при выборе ограниченного набора результатов.
Масштабируемость	Архитектура системы должна поддерживать возможность горизонтального увеличения вычислительных ресурсов без изменения основных компонентов.
Отказоустойчивость	Система должна обеспечивать коэффициент доступности не менее 99,9 % за счёт механизмов восстановления и избыточности.
Безопасность	Система должна реализовывать проверку подлинности пользователей и защиту данных в процессе взаимодействия.
Удобство использования (UX)	Пользовательский интерфейс должен корректно отображаться на устройствах с различными разрешениями экранов и обеспечивать интуитивное взаимодействие с системой.
Конфиденциальность	Обработка данных должна соответствовать действующему законодательству в области защиты персональной информации.

Заданные нефункциональные требования формируют рамочные условия для проектирования и эксплуатации системы мультимодального поиска, определяя допустимые значения ключевых метрик и ограничений. На их основе могут разрабатываться сценарии нагрузочного, отказоустойчивого и приёмочного тестирования, а также критерии оценки готовности программного решения к внедрению в целевую инфраструктуру. Кроме того, они служат инструментом согласования ожиданий между разработчиками,

заказчиком и потенциальными пользователями, обеспечивая единое понимание требуемого уровня качества системы.

3.4. Архитектурная модель системы

Архитектурная модель системы определяет состав программных компонентов, их функции и способы взаимодействия в рамках единой логики (пайплайна) обработки пользовательских запросов. Для предлагаемого решения используется многослойная архитектура, разделяющая клиентский интерфейс, серверную обработку запросов, модуль поиска и модуль генерации ответов. Такой подход повышает гибкость разработки, облегчает масштабирование отдельных компонентов и обеспечивает независимость модулей при внесении изменений.

На уровне клиентского интерфейса реализуется взаимодействие пользователя с системой: формирование запросов, отображение результатов поиска и визуализация дополнительных данных. Данный уровень обеспечивает передачу запросов на сервер, обработку ответов и управление состоянием пользовательского интерфейса. Коммуникация осуществляется по протоколу HTTP или WebSocket, что позволяет реализовать как синхронные, так и интерактивные сценарии работы.

Серверный уровень отвечает за маршрутизацию запросов, выполнение бизнес-логики, обращение к внешним сервисам и контроль потоков обработки данных. На данном уровне формируются запросы к модулю поиска и модулю генерации, обеспечивается валидация входных данных и управление очередностью операций. Серверный слой выступает связующим звеном между компонентами архитектуры, обеспечивая унифицированный интерфейс взаимодействия и независимость клиентской части от внутренних механизмов обработки.

Модуль поиска отвечает за преобразование входных данных пользователя в формат, пригодный для семантического сопоставления, выполнение векторного сравнения и получение релевантных результатов из хранилища данных. На данном этапе обеспечивается поиск ближайших соседей и отбор документов, соответствующих контексту запроса. Результаты поиска передаются в модуль генерации, который формирует финальный ответ с использованием генеративной модели. Такой двухступенчатый подход соответствует принципам Retrieval-Augmented Generation и позволяет уменьшить вероятность генерации недостоверных ответов за счёт учета информации из внешних источников.

Модуль хранения данных реализует размещение векторных представлений и метаданных, а также обеспечивает поддержку операций добавления, удаления и обновления записей. Отдельный уровень отвечает за масштабирование и распределённую обработку данных, что позволяет выполнять вычисления параллельно и повышает производительность системы при увеличении количества запросов. Архитектурный подход обеспечивает изоляцию функциональных модулей, облегчает сопровождение решения и позволяет дополнительно интегрировать механизмы мониторинга, журналирования и автоматизации развертывания.

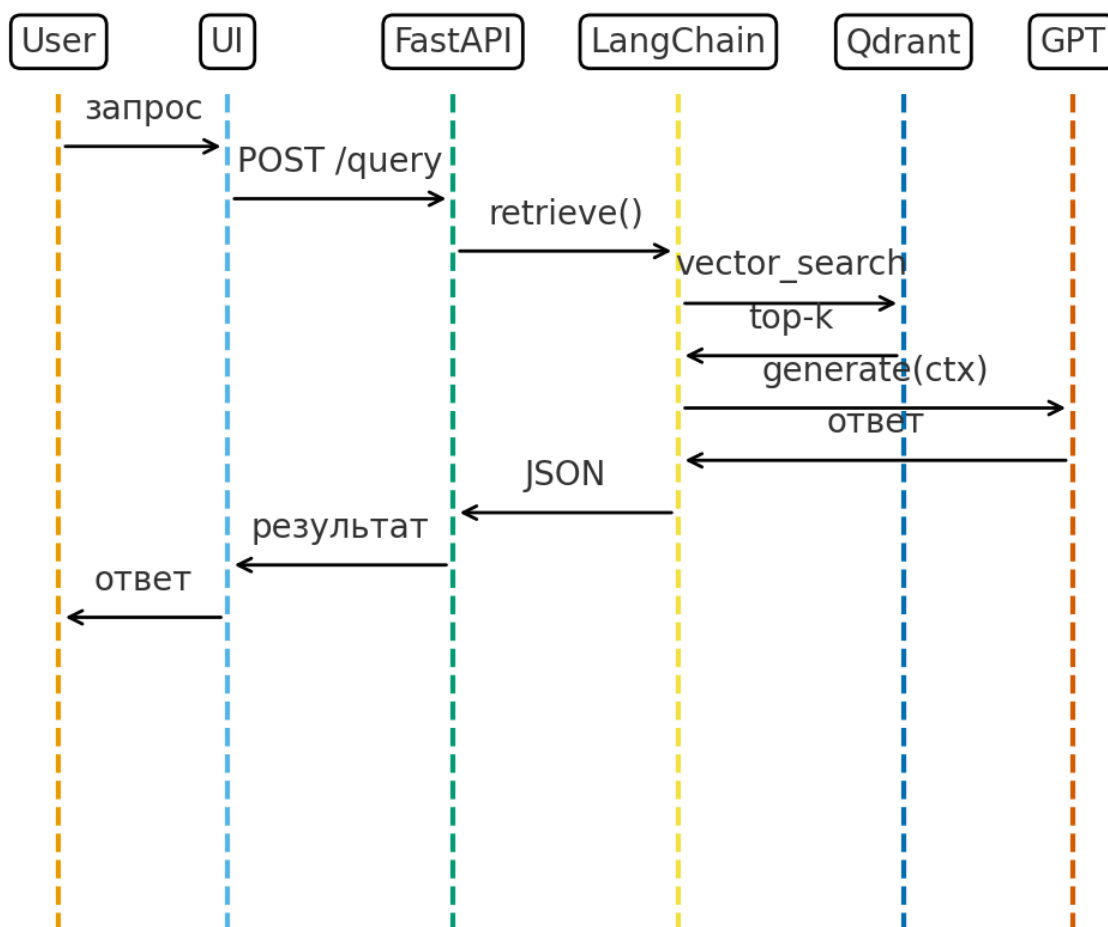


Рисунок 3.1 — Схема пайплайна обработки запроса

Представленная диаграмма последовательности демонстрирует поток обработки пользовательского запроса: от интерфейса UI через FastAPI и RAG-цепочку LangChain к векторному хранилищу Qdrant и языковой модели GPT. Схема наглядно показывает обмен сообщениями между компонентами системы и этапы формирования итогового ответа пользователю.

Заключение

Проведенное исследование достигло своей цели — проведён комплексный анализ методов реализации мультимодального поиска на основе RAG-архитектуры, исследованы технологические подходы к хранению векторных данных, получению эмбедингов и генерации естественно-языковых ответов, а также обоснован выбор практических инструментов разработки серверной и клиентской части. Результаты выполнения поставленных задач подтверждают достижение цели работы.

В ходе анализа существующих методов семантического и полнотекстового поиска были выявлены структурные ограничения традиционных алгоритмов, основанных на статистических метриках подобия документов. Показано, что с ростом объёмов данных и усложнением их модальной природы применение классических подходов не обеспечивает требуемого уровня релевантности, что обусловило необходимость использования методов векторизации и векторного поиска.

При сравнении специализированных систем хранения эмбедингов установлены ключевые преимущества современных векторных баз данных. Анализ характеристик Qdrant и pgvector позволил выявить закономерность: специализированные решения, основанные на графовых алгоритмах (HNSW), демонстрируют более высокую скорость поиска и лучше адаптированы к мультимодальным запросам с гибридной фильтрацией метаданных. Это стало основанием выбора Qdrant в качестве основного векторного хранилища для разработки программного решения.

Исследование подходов к интеграции генеративных моделей и векторного поиска позволило определить оптимальную технологическую архитектуру, включающую LangChain для формирования RAG-цепочек, FastAPI для реализации серверной логики и Ray для организации

распределённых вычислений. Показано, что использование данных инструментов обеспечивает сокращение времени отклика, повышение устойчивости системы и возможность горизонтального масштабирования при увеличении количества запросов и объёмов данных.

Дополнительно проведён анализ вспомогательных технологий, обеспечивающих масштабируемость и сопровождаемость системы.

В заключение, в ходе исследования была поставлена цель — проанализировать методы и технологии обработки мультимодальных данных и разработка требований к веб-приложению, предназначенному для мультимодального поиска и анализа данных на основе RAG-архитектуры и модулей генеративных языковых моделей. Для её достижения были выполнены следующие задачи:

1. Проведен обзор отечественных и зарубежных публикаций, посвящённых вопросам мультимодального поиска, векторного представления данных и применения RAG-архитектур;
2. Выполнен сравнительный анализ современных технологий разработки серверных и клиентских приложений для реализации систем интеллектуального поиска;
3. Сформированы функциональные и нефункциональные требования к проектируемому веб-приложению;
4. Определены архитектурные особенности предлагаемого решения, включая структуру данных, интеграционные механизмы и критерии масштабируемости.

Список использованных источников

- [1] Большие языковые модели (LLM) // Yandex Cloud: Глоссарий. URL: <https://yandex.cloud/ru/docs/glossary/llm> (дата обращения: 08.10.2025).
- [2] Визуально-языковые модели (VLM) // Yandex Cloud: Блог. URL: <https://yandex.cloud/ru/blog/vlm-visual-language-models> (дата обращения: 08.10.2025).
- [3] What is Retrieval-Augmented Generation (RAG)? // AWS. URL: <https://aws.amazon.com/ru/what-is/retrieval-augmented-generation/> (дата обращения: 08.10.2025).
- [4] Ретриверы // Sber: GigaChain Documentation. URL: <https://developers.sber.ru/docs/ru/gigachain/tutorials/retrievers> (дата обращения: 09.10.2025).
- [5] Embeddings в GigaChat // Sber: GigaChat Documentation. URL: <https://developers.sber.ru/docs/ru/gigachat/guides/embeddings> (дата обращения: 09.10.2025).
- [6] Multimodal RAG // IBM Think. URL: <https://www.ibm.com/think/topics/multimodal-rag> (дата обращения: 09.10.2025).
- [7] An Easy Introduction to Multimodal Retrieval-Augmented Generation // NVIDIA Developer Blog. URL: <https://developer.nvidia.com/blog/an-easy-introduction-to-multimodal-retrieval-augmented-generation/> (дата обращения: 09.10.2025).
- [8] Multimodal RAG Using Document Retrieval and VLMs // Hugging Face. URL: https://huggingface.co/learn/cookbook/multimodal_rag_using_document_retrieval_and_vlms (дата обращения: 09.10.2025).

- [9] Gemini Multimodal RAG // LangChain OpenTutorial. URL: <https://langchain-opentutorial.gitbook.io/langchain-opentutorial/19-cookbook/06-multimodal/10-geminimultimodalrag> (дата обращения: 09.10.2025).
- [10] AIJ — портал об искусственном интеллекте // AIJ.ru. URL: <https://aij.ru/> (дата обращения: 09.10.2025).
- [11] Новости компании «Яндекс» // Yandex. URL: <https://yandex.ru/company/news/04-25-02-2025> (дата обращения: 09.10.2025).
- [12] Embeddings в Yandex AI Studio // Yandex Cloud. URL: <https://yandex.cloud/ru/docs/ai-studio/concepts/embeddings> (дата обращения: 09.10.2025).
- [13] Embeddings // OpenAI Platform. URL: <https://platform.openai.com/docs/guides/embeddings> (дата обращения: 09.10.2025).
- [14] CLIP Model Documentation // Hugging Face Transformers. URL: https://huggingface.co/docs/transformers/model_doc/clip (дата обращения: 25.10.2025).
- [15] BLIP-2: Scalable Pre-training of Multimodal Foundation Models for the World's First Open-source Multimodal Chatbot // The 360 Blog. URL: <https://www.salesforce.com/blog/blip-2/> (дата обращения: 25.10.2025).
- [16] What is CI/CD? // GitLab. URL: <https://about.gitlab.com/topics/ci-cd/> (дата обращения: 25.10.2025).
- [17] Python 3 Documentation // Python.org. URL: <https://docs.python.org/3/> (дата обращения: 26.10.2025).
- [18] FastAPI Documentation // FastAPI. URL: <https://fastapi.tiangolo.com/> (дата обращения: 26.10.2025).

- [19] REST API // Yandex Cloud: Глоссарий. URL: <https://yandex.cloud/ru/docs/glossary/rest-api> (дата обращения: 26.10.2025).
- [20] Материал о REST API // Habr: Песочница. URL: <https://habr.com/ru/sandbox/171066/> (дата обращения: 26.10.2025).
- [21] Flask Documentation // Flask. URL: <https://flask.palletsprojects.com/en/stable/> (дата обращения: 26.10.2025).
- [22] Django REST framework — API guide // Django REST framework. URL: <https://www.django-rest-framework.org/> (дата обращения: 26.10.2025).
- [23] Статья о FastAPI и асинхронных сервисах // Habr. URL: <https://habr.com/ru/articles/776538/> (дата обращения: 26.10.2025).
- [24] LangChain Documentation // LangChain. URL: <https://docs.langchain.com/> (дата обращения: 26.10.2025).
- [25] API Reference // OpenAI API. URL: <https://platform.openai.com/docs/api-reference/introduction> (дата обращения: 10.11.2025).
- [26] Hugging Face — платформа для машинного обучения // Hugging Face. URL: <https://huggingface.co/> (дата обращения: 10.11.2025).
- [27] What is LangChain? // AWS. URL: <https://aws.amazon.com/ru/what-is/langchain/> (дата обращения: 10.11.2025).
- [28] Qdrant Documentation // Qdrant. URL: <https://qdrant.tech/documentation/> (дата обращения: 10.11.2025).
- [29] pgvector Vector Store Integration // LangChain. URL: <https://docs.langchain.com/oss/python/integrations/vectorstores/pgvector> (дата обращения: 10.11.2025).

- [30] What is k-Nearest Neighbors (kNN)? // IBM Think. URL: <https://www.ibm.com/think/topics/knn> (дата обращения: 11.11.2025).
- [31] PostgreSQL Official Site // PostgreSQL. URL: <https://www.postgresql.org/> (дата обращения: 11.11.2025).
- [32] gRPC — A high performance, open source universal RPC framework // gRPC. URL: <https://grpc.io/> (дата обращения: 11.11.2025).
- [33] What is Structured Query Language (SQL)? // IBM Think. URL: <https://www.ibm.com/think/topics/structured-query-language> (дата обращения: 11.11.2025).
- [34] Ray Documentation // Ray. URL: <https://docs.ray.io/en/latest/index.html> (дата обращения: 12.11.2025).
- [35] Celery Documentation // Celery. URL: <https://docs.celeryq.dev/en/stable/> (дата обращения: 13.11.2025).
- [36] Apache Spark Official Site // Apache Spark. URL: <https://spark.apache.org/> (дата обращения: 13.11.2025).
- [37] Docker Documentation // Docker. URL: <https://docs.docker.com/> (дата обращения: 13.11.2025).
- [38] Kubernetes Documentation // Kubernetes. URL: <https://kubernetes.io/docs/home/> (дата обращения: 13.11.2025).
- [39] TypeScript Documentation // TypeScript. URL: <https://www.typescriptlang.org/docs/> (дата обращения: 14.11.2025).
- [40] JavaScript — MDN Web Documentation // Mozilla Developer Network. URL: <https://developer.mozilla.org/en-US/docs/Web/JavaScript> (дата обращения: 14.11.2025).

- [41] Nuxt 4 Documentation. Getting Started // Nuxt. URL: <https://nuxt.com/docs/4.x/getting-started/introduction> (дата обращения: 14.11.2025).
- [42] Vue.js Guide. Introduction // Vue.js. URL: <https://vuejs.org/guide/introduction> (дата обращения: 14.11.2025).
- [43] Статья о Nuxt и SSR-приложениях // Habr. URL: <https://habr.com/ru/articles/798585/> (дата обращения: 19.11.2025).
- [44] Статья о Vue 3 и композиционном API // Habr. URL: <https://habr.com/ru/articles/568280/> (дата обращения: 19.11.2025).
- [45] Install Tailwind CSS with Vite // Tailwind CSS. URL: <https://tailwindcss.com/docs/installation/using-vite> (дата обращения: 19.11.2025).
- [46] Vite Documentation // Vite. URL: <https://vite.dev/> (дата обращения: 19.11.2025).
- [47] Webpack Documentation // Webpack. URL: <https://webpack.js.org/> (дата обращения: 20.11.2025).
- [48] ESLint Documentation // ESLint. URL: <https://eslint.org/docs/latest/> (дата обращения: 20.11.2025).
- [49] Prettier Documentation // Prettier. URL: <https://prettier.io/docs/> (дата обращения: 20.11.2025).
- [50] UV Documentation // Astral. URL: <https://docs.astral.sh/uv/> (дата обращения: 20.11.2025).
- [51] pip User Guide // Python Package Index. URL: <https://pip.pypa.io/en/stable/> (дата обращения: 20.11.2025).

- [52] Poetry Documentation // Python Poetry. URL: <https://python-poetry.org/docs/> (дата обращения: 20.11.2025).
- [53] Bun Package Manager // Bun. URL: <https://bun.com/package-manager> (дата обращения: 23.11.2025).
- [54] Node.js — JavaScript runtime // Node.js. URL: <https://nodejs.org/en> (дата обращения: 23.11.2025).
- [55] npm Documentation // npm. URL: <https://www.npmjs.com/> (дата обращения: 23.11.2025).
- [56] Yarn — Package Manager // Yarn. URL: <https://yarnpkg.com/> (дата обращения: 23.11.2025).
- [57] Prometheus Documentation. Introduction // Prometheus. URL: <https://prometheus.io/docs/introduction/overview/> (дата обращения: 29.11.2025).
- [58] Grafana Documentation // Grafana Labs. URL: <https://grafana.com/docs/> (дата обращения: 29.11.2025).
- [59] pre-commit — A framework for managing and maintaining multi-language pre-commit hooks // pre-commit. URL: <https://pre-commit.com/> (дата обращения: 29.11.2025).
- [60] Material for MkDocs Documentation // MkDocs-Material. URL: <https://squidfunk.github.io/mkdocs-material/> (дата обращения: 29.11.2025).
- [61] Markdown Guide // Markdown Guide. URL: <https://www.markdownguide.org/> (дата обращения: 29.11.2025).
- [62] Mermaid — Diagramming and charting tool // Mermaid. URL: <https://mermaid.js.org/> (дата обращения: 04.12.2025).

[63] Статья о документации и архитектуре на MkDocs // Habr. URL: <https://habr.com/ru/companies/tochka/articles/693834/> (дата обращения: 04.12.2025).